

1. Interpretazione dei dati:

Processo	Tempo di esecuzione	Tempo di attesa	Tempo di esecuzione dopo attesa
P1	3 secondi	2 secondi	1 secondo
P2	2 secondi	1 secondo	-
P3	1 secondi	-	-
P4	4 secondi	1 secondo	-

Solo **P1** ha una vera fase "dopo l'attesa": esegue 3s, poi aspetta I/O 2s, poi torna in CPU per 1s. Gli altri (P2, P4) eseguono, poi aspettano I/O ma non hanno più bisogno della CPU dopo (l'attesa è probabilmente per terminare un'operazione di output finale, non seguita da altro calcolo). P3 non ha alcuna attesa I/O.

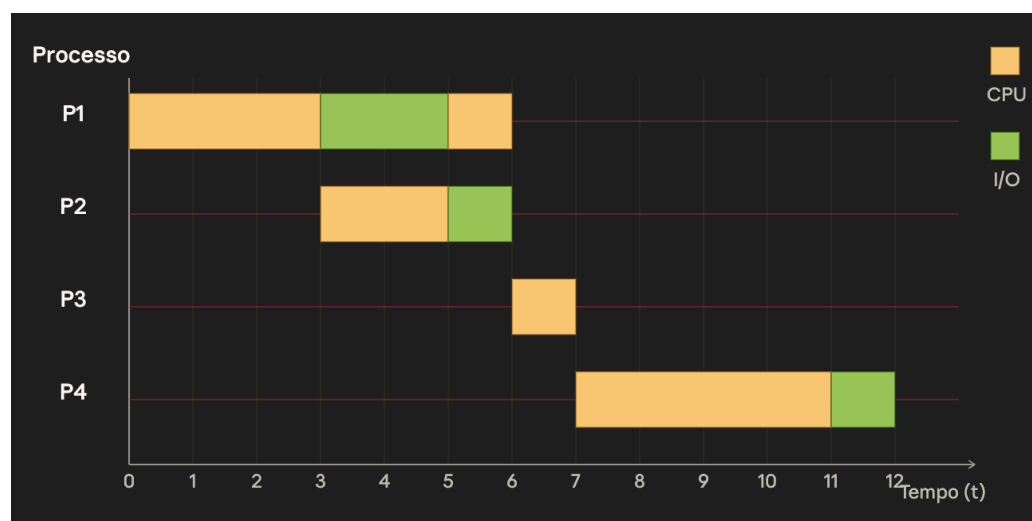
2. Perché un sistema multi-tasking è la scelta più efficace:

Se i processi fossero eseguiti in **mono-tasking puro** (un processo tiene la CPU anche mentre aspetta I/O), il tempo totale sarebbe semplicemente la somma di tutto: $3+2+1$ (P1) + $2+1$ (P2) + 1 (P3) + $4+1$ (P4) = **15 secondi**, con la CPU che resta inutilizzata per 4 secondi totali (i tempi di attesa) mentre potrebbe lavorare su altri processi.

In un sistema **multi-tasking**, possiamo "riempire" i buchi di attesa di P1 con il lavoro di P2, P3, P4. Questo è esattamente il principio di **sovrapposizione tra CPU e I/O**.

3. Costruzione del diagramma:

Procediamo tempo per tempo, $P1 \rightarrow P2 \rightarrow P3 \rightarrow P4$, assegnando la CPU al primo processo pronto in coda quando quella in esecuzione si ferma:



4. Riepilogo della linea temporale

P2 termina a $t=6$ (il suo I/O finisce a $t=6$, e non avendo un burst successivo da svolgere, il processo è completo). **P1 termina a $t=6$, P3 a $t=7$, P4 a $t=12$.**

5. Perché questo è il metodo più efficace

Confrontando con il mono-tasking puro (15 secondi totali), qui il tempo totale per completare tutti e 4 i processi è **12 secondi**, perché:

- Durante l'attesa I/O di P1 ($t=3 \rightarrow 5$), la CPU non resta inattiva: lavora su P2.
- L'unico "buco" di CPU inutilizzata è l'ultimo secondo ($t=11 \rightarrow 12$), dove P4 è in I/O finale e non c'è più nessun processo pronto in coda.

Questo è il principio del multi-tasking : quando un processo entra in stato di attesa (per I/O), lo scheduler non lascia la CPU ferma, ma la cede a un altro processo pronto in coda. È esattamente l'evoluzione da mono-tasking a multi-tasking.